



API testing interview questions



The Postman Team

August 16, 2023

So, you have a big job interview coming up, and you know they're going to grill you on your [API testing](#) knowledge. First of all, congratulations! This is an exciting moment, regardless of whether you're a recent graduate or a seasoned developer. This article will help you brush up on your answers to some common API testing questions, so you can arrive at your interview feeling prepared, confident, and ready to show off your skills.

1. What is an API?

An [API](#), or application programming interface, is a set of protocols that enable different software components to communicate and transfer data. Developers use private, partner, and public APIs to bridge the gaps between small, discrete chunks of code in order to create applications that are powerful, resilient, secure, and able to meet user needs.

You most likely have some familiarity with APIs if you've gotten to this stage of the interview process. Still, it can be challenging to clearly articulate what APIs are and how they work. It's therefore important to come prepared with a solid answer to this seemingly simple question.

2. What is the difference between manual API testing and automated API testing?

Manual API testing involves sending a request to an API in an ad-hoc manner and inspecting the response. Manual testing is exploratory in nature, and it is often performed during the development process in order to validate assumptions and gain important insights that can inform code-level decisions.

Automated API testing, on the other hand, involves running scripts with a testing tool at predetermined intervals or in response to events. [API test automation](#) helps teams expand their test coverage in a systematic way and can be employed at every stage of the development lifecycle, including from within CI/CD pipelines. Automated API testing is highly scalable, which makes it well-suited for teams that ship updates frequently.

3. What are the different types of API testing?

There are numerous types of API tests, and each one plays an important role in validating the functionality, health, and performance of an API. Some of the most common types include:

- ✦ **Contract testing:** An API contract acts as the single source of truth for what each request and response should look like—and forms the basis of service-level agreements (SLAs) between API producers and consumers. API contract testing helps ensure that new releases don't violate the contract by checking the content and format of requests and responses.
- ✦ **Unit testing:** Unit testing focuses on testing an API's endpoints, functions, methods, and classes in isolation in order to confirm that each unit is working as expected. For instance, unit tests may validate that an endpoint handles optional parameters correctly, or that it returns the appropriate error message when sent an invalid request.
- ✦ **Performance testing:** Performance testing measures response times, throughput, resource utilization, and scalability under varying load conditions in order to identify bottlenecks and ensure the API can reliably meet performance requirements.
- ✦ **Security testing:** [API security](#) testing helps identify and address potential vulnerabilities in the API. It involves scanning for common security risks like authentication and authorization vulnerabilities, injection attacks, cross-site scripting, and sensitive data exposure.
- ✦ **End-to-end testing:** End-to-end tests are used to validate key user journeys that may involve multiple endpoints and APIs. End-to-end API testing involves chaining requests together and confirming that each one is working properly, which helps teams surface issues in complex workflows.

4. What are some steps you can take to plan and implement an effective API testing strategy?

It's essential to create a comprehensive API tes
This involves making sure you have a thorough
combinations of inputs and outputs, and organ
complete coverage while making your test suite

5. What is a testing enviro

An API testing environment contains the resour
environment, enabling teams to test an API unc
production in order to ensure that live systems and data are not affected, and they should also have appropriate security

We use cookies on your device to enhance your navigation experience, analyze usage to improve our site, and customize our marketing efforts. You can learn more about cookies in our [Cookie Notice](#).

[Manage Cookies](#)[Reject All Cookies](#)[Accept All Cookies](#)

APIs. End-to-end API testing involves chaining requests together and confirming that each one is working properly, which helps teams surface issues in complex workflows.

4. What are some steps you can take to plan and implement an effective API testing strategy?

It's essential to create a comprehensive API testing plan—whether you intend to run your tests manually or automatically. This involves making sure you have a thorough understanding of the API's requirements, identifying all possible combinations of inputs and outputs, and organizing your tests into categories. These steps will help ensure you get complete coverage while making your test suite easier to manage.

5. What is a testing environment?

An API testing environment contains the resources, variables, data, and infrastructure that collectively mimic the production environment, enabling teams to test an API under real-world conditions. Testing environments should be fully isolated from production in order to ensure that live systems and data are not affected, and they should also have appropriate security measures in place, including access controls, data encryption, and secure network configurations.

6. How do you handle error responses or exceptions in API testing?

Proper [API error handling](#) contributes to a positive user experience, and an effective API test suite should verify that the API handles errors and exceptions appropriately. One approach is to create negative test cases that include invalid data in API requests, which helps validate that the API responds with the correct error codes, messages, and any additional information. It's also important to include positive test cases that confirm error responses follow established patterns, such as using standard HTTP status codes like 400 for bad requests or 500 for internal server errors.

7. How do you test APIs for performance and scalability?

An API test suite should do more than confirm the API works as expected. It should also verify that it can maintain appropriate response times under varying loads. This type of testing, which is called performance testing, enables teams to monitor an [API's latency](#), resource usage, and error rate while it responds to simulated traffic. Some API performance testing tools can be configured to increase simulated traffic over time, which makes it easier to spot performance thresholds and bottlenecks.

8. What is API versioning and how does it impact testing?

[API versioning](#) provides a framework for teams to safely manage and roll out changes to an API while keeping consumers informed. Test cases should cover scenarios specific to each version, ensuring that changes in functionality or behavior are thoroughly tested and that backward compatibility is maintained. Full test coverage for each version is especially important when multiple versions are in circulation at the same time, as a test that passes for one version may fail for another.

9. What is dynamic input data, and why is it important for API testing?

Dynamic input data is programmatically generated data that is used as input during the testing process. Testing an API with dynamic input data allows testers to explore a wider range of scenarios than they can with static test data. Static data might not accurately represent the complexity of real-world API usage, and it is likely to be influenced by the biases of the test writer, who might not be aware of potential edge cases.

10. How do you test API workflows that involve dependencies, such as databases or external services?

Many API workflows often integrate with external services, databases, or other APIs, and these dependencies need to be accounted for in tests to ensure full coverage. This can be challenging, as dependent services—especially those that are managed by a third-party—may be difficult to set up for testing purposes. Techniques such as mocking or stubbing allow testers to mimic the behavior and responses of the actual dependencies so that testing can be performed independently of the external systems—without sacrificing coverage.

11. Can you talk about a time when you encountered a challenging API testing scenario and how you resolved it?

This question is intended to gauge your familiarity with complex configurations and edge cases that are likely to exist in production-level APIs. Even if you don't have much real-world API testing experience, you might talk about how you would approach testing an API that has complex [API authentication](#) and authorization mechanisms. For instance, you could discuss how tools like Postman enable you to maintain a dedicated testing environment and handle authentication headers, tokens, and user credentials.

12. What are the most important best practices for API testing?

There are several best practices that can help teams design and implement an effective API testing strategy, including:

- ✦ **Creating a dedicated testing environment:** Tests should be executed in a dedicated environment in order to contain any issues and avoid user-facing downtime. This environment should mimic the production environment, but it should include mock data that can be safely manipulated.

- ✦ **Automating API tests:** Test automation enables teams to execute tests in a continuous, predictable way throughout

- Collection Runner: The Collection Runner feature allows you to execute multiple requests in a collection as a test suite, enabling automation and bulk testing.

4. How do you handle authentication and authorization in Postman?

Postman provides multiple ways to handle authentication and authorization in API requests. Some commonly used methods include:

- Basic Authentication: You can include the username and password in the request headers using the "Authorization" header.
- Token-based Authentication: Postman allows you to include tokens (such as JWT or OAuth) in the request headers or as query parameters.
- API Key: If an API requires an API key, you can pass it as a request header or query parameter.
- OAuth 2.0: Postman has built-in OAuth 2.0 support, allowing you to configure and authenticate using various OAuth flows, such as Authorization Code or Client Credentials.
- Custom Authentication: Postman's scripting capabilities enable you to implement custom authentication mechanisms by modifying request headers or using specialized libraries.

5. Can you describe the process of creating and running automated tests in Postman?

To create and run automated tests in Postman, follow these steps:

- Create a new request in Postman or open an existing one.
- Write test scripts using JavaScript to validate the API response, assert specific conditions, or extract data.
- Save the request in a collection.
- Go to the "Collections" tab and click on the three-dot menu next to the collection name.
- Select "Edit" to open the collection, then click on the "Tests" tab.
- Write and organize test scripts for the entire collection or individual requests within the collection.
- Save the collection.
- To run the automated tests, go back to the "Collections" tab and click on the arrow icon next to the collection name.
- Choose the desired test environment, if applicable.
- Click on "Run" to execute the tests in the collection.
- Postman will run the requests and display the test results, highlighting any failures or errors encountered during the execution.

Postman Interview Questions for Freshers

6. What is Postman?

Postman is a collaboration platform for API development. It is a recognized API client that enables you to organize the creation, division, testing, and documentation of APIs. We can send HTTP/s requests to a repair and receive their responses using the Postman tool. It will enable us to approve the service's uptime and functionality.

7. What is a collection in Postman?

In Postman, a collection allows you to group related requests. You can also use it to organize the requests into folders systematically.

8. Why do we use Postman?

Following are some of the most important reasons for using Postman:

- It is software that aids with API testing and is available for free use

Most Asked Postman Interview Questions

1. Explain the purpose and benefits of using Postman for API testing?

Postman is a powerful tool for API testing that offers numerous benefits. Its purpose is to simplify and streamline the process of testing APIs. Some benefits of using Postman for API testing include:

- **Easy API Exploration:** Postman provides a user-friendly interface for creating and sending requests to APIs, allowing you to explore and understand their functionalities.
- **Efficient Testing:** With Postman, you can quickly test various scenarios by sending different types of requests (GET, POST, PUT, DELETE) and examining the responses.
- **Request and Response Validation:** Postman allows you to validate API responses, ensuring they meet the expected criteria and identifying any errors or inconsistencies.
- **Environment Management:** Postman lets you create and manage environments, making it convenient to switch between different setups, such as development, staging, and production.
- **Collaboration and Sharing:** Postman facilitates teamwork by enabling the sharing of collections, requests, and test suites, fostering collaboration among developers, testers, and other stakeholders.

2. How do you create and send a request in Postman?

To create and send a request in Postman, follow these steps:

- Open Postman and create a new request by selecting the appropriate HTTP method (e.g., GET, POST) from the dropdown menu.
- Enter the request URL in the address bar.
- Add any required headers, query parameters, or request body data, depending on the API endpoint's requirements.
- Click the "Send" button to send the request to the API server.

Postman will display the response, including the status code, headers, and body, allowing you to analyze the results of your request.

3. What are some key features of Postman that you find most useful in your testing workflow?

Some key features of Postman that are often found useful in testing workflows include:

- **Collections:** It allows you to organize and group related requests, making managing and executing test suites easier.
- **Variables and Environments:** Postman allows you to define variables and environments, which help parameterize requests and manage different testing environments.
- **Test Scripts:** Postman's test scripts, written in JavaScript, enable you to automate assertions, extract data from responses, and perform complex validations.
- **Pre-request Scripts:** These scripts allow you to perform actions before sending a request, such as setting up dynamic data or configuring authentication.
- **Request History:** Postman keeps a history of previously sent requests, allowing you to revisit and retest them without re-entering the details.
- **Collection Runner:** The Collection Runner feature allows you to execute multiple requests in a collection as a test suite, enabling automation and bulk testing.

4. How do you handle authentication and authorization in Postman?

Postman provides multiple ways to handle authentication and authorization in API requests. Some commonly used methods include:

- **Basic Authentication:** You can include the username and password in the request headers using the "Authorization" header.

Blog

approach testing an API that has complex [API authentication](#) and authorization mechanisms. For instance, you could discuss how tools like Postman enable you to maintain a dedicated testing environment and handle authentication headers, tokens, and user credentials.

12. What are the most important best practices for API testing?

There are several best practices that can help teams design and implement an effective API testing strategy, including:

- ✦ **Creating a dedicated testing environment:** Tests should be executed in a dedicated environment in order to contain any issues and avoid user-facing downtime. This environment should mimic the production environment, but it should include mock data that can be safely manipulated.
- ✦ **Automating API tests:** Test automation enables teams to execute tests in a continuous, predictable way throughout the [API lifecycle](#). Teams can use a variety of tools to automate their tests to execute at specific times, specific frequencies, or in CI/CD pipelines.
- ✦ **Writing reusable subtests:** Every [API endpoint](#) is different and should be tested with its own custom logic. That being said, there may be certain rules that are universally applicable, such as a requirement that all responses be formatted in JSON. Teams can create subtests for these use cases that can be reused throughout their test suite, which reduces the risk of human error.
- ✦ **Implementing a combination of functional, performance, and security tests:** It's not enough to simply confirm that an API's endpoints are working as expected. Teams also need to monitor response times and error rates—and run regular checks for potential security vulnerabilities.

13. Which tools and frameworks do you use for API testing?

There are many tools and frameworks that can help you test your APIs. The Postman [API Platform](#) is the gold standard for API testing, and any interviewer will be impressed if you're able to talk about its API testing features in detail. For instance, you might mention that:

- ✦ Postman supports testing across a wide range of API architectures, including [REST](#), GraphQL, [SOAP](#), gRPC, and Websocket.
- ✦ Postman users can quickly author test scripts with a library of pre-configured [code snippets](#), and they can execute these tests manually, on a schedule, or from within their CI/CD pipeline of choice.
- ✦ Postman lets users store [variables](#) at the [environment level](#), which can be used to create a dedicated testing environment.
- ✦ Postman's [Collection Runner](#) allows teams to chain tests together in order to validate complex, end-to-end workflows.
- ✦ Postman's [API performance testing](#) feature enables users to confirm their API can handle the expected load—and helps them identify any bottlenecks as traffic volume increases over time.

This is just a sampling of what Postman has to offer. If you'd like to get some hands-on experience with API testing before your interview, [try Postman for free](#).

Did we miss your favorite API testing interview question? If so, leave it in the comments below!



Tags: [API 101](#) [API Testing](#) [Tutorials](#)



The Postman Team



Postman is the single platform for designing, building, and scaling APIs—together. Join over 40 million users who have consolidated their workflows and leveled up their API game—all in one powerful platform.

[View all posts by The Postman Team →](#)

What do you think about this topic? Tell us in a comment below.

Comment

Your name

Your email

Write a public comment

Postman tests are run using [JavaScript](#).

22. What is a Postman Collection?

We can group several needs using a Postman Collection. It merely enables us to organize the requests into folders.

23. What do you understand by the term Postman Collection runners?

Data-driven testing is carried out by using a Postman collection runner. API requests are collected for numerous iterations with various data sets.

24. What is Postman cloud's purpose if we work in a company? Why?

A Postman cloud is a shared repository where businesses can access Postman collections. Work can be immediately saved in the Postman cloud after logging in. Data and collections are accessible to team members from any location.

25. Why is it not preferred to save work in Postman cloud?

Since company work cannot be disclosed and must be kept confidential, it is not advised or desired to save your work in the Postman cloud. Since Postman requires sign-in, saving your work there could lead to a security risk.

Learn 15+ In-Demand Tools and Skills!

Automation Testing Masters Program

[EXPLORE PROGRAM](#)

10M New Jobs in 2023

Learn **15+** In-Demand Tools

26. How will you log variable values in Postman?

You may use the following command in Postman to log the variable values on the console:

```
console.log(pm.variables.get("variable_name"));
```

27. How do you access postman variables?

You may access a Postman variable using the variable name `{{var}}`.

28. What are the various authorization methods provided by Postman?

Postman provides the following API request authorization options:

- API Key
- Basic auth
- Digest auth
- Hawk Authentication
- OAuth 1.0
- OAuth 2.0
- Bearer Token
- NTLM Authentication
- AWS Signature

29. What are the different types of API requests supported in Postman?

environments: global and local. They define the variable's scope so you can use it in requests.

15. What are the core components of an HTTP request?

An HTTP request is made up of five major components:

- HTTP methods: A collection of request techniques used to carry out particular actions on resources (GET, PUT, POST, DELETE)
- URI (Uniform Resource Identifier): locates a resource.
- Version of HTTP (example- HTTP v1.1)
- Content-Type: application/JSON, Content-Length: 511) Request Headers
- Payload: The message content is contained in the request body.

Prepare Yourself to Answer All Questions!

Automation Testing Masters Program

EXPLORE PROGRAM

10M New Jobs in 2023

Learn **15+** In-Demand Tools



16. State the Core Components of an HTTP Response?

Four essential components are present in each HTTP response:

- According to the HTTP request's resource, the server's status is indicated by the Status/Response Code. Examples include 404 (resource not found) and 200 (response ok).
- HTTP Version - Indicates the version of HTTP being used, such as HTTP v1.1.
- The response header is where key-value pairs of metadata for the HTTP response message are stored—for instance, content length, content type, response date, and server type.
- The response body indicates response message content or resource representation.

17. What is GUID?

Global Unique Identifier is referred to as GUID. It consists of hexadecimal digits separated by hyphens. This Postman identifier GUID fulfills the goal of uniqueness.

18. What is the HTTP response code for a POST request with incorrect parameters?

The correct response code for a request with incorrect parameters is 400 Bad Request.

19. Can local variables be imported in Postman Monitors?

Yes, you can import local variables into Postman Monitors. However, [global variables](#) cannot get imported into Postman Monitors.

20. How can you iterate a request 100 times in Postman?

With the help of Collection Runner, Postman allows us to iterate a request 100 times.

21. Which programming language is used for Postman tests?

Postman tests are run using [JavaScript](#).

22. What is a Postman Collection?

We can group several needs using a Postman Collection. It merely enables us to organize the requests into folders.

In Postman, a collection allows you to group related requests. You can also use it to organize the requests into folders systematically.

8. Why do we use Postman?

Following are some of the most important reasons for using Postman:

- It is software that aids with API testing and is available for free use
- It aids in managing the complete API lifecycle
- It provides Runtime Service to assist with managing API collections, workspaces, environments, and many examples
- Additionally, Postman may get integrated with CI/CD programs like Circle CI, [Jenkins](#), etc
- It offers a sizable community forum where you can quickly find solutions to any technical problems you run across while using the tool

9. What is an API?

Application Programming Interface is referred to as API. Technically speaking, an API is a collection of steps, functions, and other points of access that a program, an operating system, a library, etc. makes available to [programmers](#) so they can use it to communicate with other software programs.

10. Name some tools used for API Testing?

The following list includes some of the tools used for API testing:

- Postman
- Tricentis Tosca
- Katalon Studio
- Apigee
- Jmeter
- SoapUI

11. What API information is exposed in Web Developer tools?

API information for request headers, response body, and response cookies is exposed in web developer tools.

12. What can we use to get API information from web developer tools into Postman?

Copy as cURL allows Postman to receive API data from web developer tools.

13. Where are query parameters stored in a GET request?

The query parameters for the GET request are saved in Postman's URL.

14. What is meant by the term environment in Postman?

An environment in Postman is a group of key-value pairs. Postman allows you to create various environments that you can switch instantly with a button. There are two different sorts of environments: global and local. They define the variable's scope so you can use it in requests.

15. What are the core components of an HTTP request?

An HTTP request is made up of five major components:

- HTTP methods: A collection of request techniques used to carry out particular actions on resources (GET, PUT, POST, DELETE)
- URI (Uniform Resource Identifier): locates a resource.
- Version of HTTP (example- HTTP v1.1)

41. When do we use global variables, collection variables, and local variables?

Global variables are all-purpose variables perfect for prototyping and quick results. They get utilized when transferring data to other requests.

You can most frequently use collection variables to store constants that don't change while the Collection is being executed. They are used for constants that remain the same throughout execution and for URLs and authentication credentials when just one environment is present.

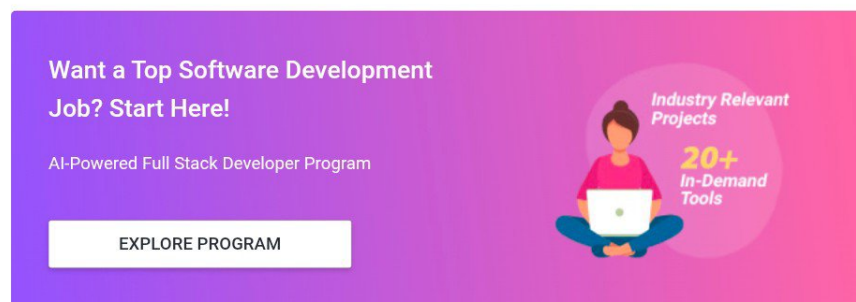
Local variables are only accessible for the duration of the request that set them or while using the Newman/Collection runner. They get used whenever you want to override all other variable scopes.

42. How do you remove local variables?

Once you run and finish the tests, the local variables are automatically eliminated.

43. What is the difference between form data and x-www-form-urlencoded?

The primary distinction between form data and x-www-form-urlencoded is that the URL is pre-arranged when sent via x-www-form-urlencoded.



Want a Top Software Development Job? Start Here!

AI-Powered Full Stack Developer Program

EXPLORE PROGRAM

Industry Relevant Projects

20+ In-Demand Tools

The advertisement features a pink and purple gradient background. On the left, there is a white button that says 'EXPLORE PROGRAM'. On the right, there is an illustration of a person sitting cross-legged and using a laptop. To the right of the person, there is a circular badge that says '20+ In-Demand Tools' and 'Industry Relevant Projects'.

44. How do you access the history of requests in Postman?

The History tab in the Postman application allows us to access the history of requests. The record will be synced amongst all your logged-in devices when we login into the Postman account. The view will load the request we saved while working on it earlier after clicking on any of the requests shown in the History tab.

Postman Interview Questions for Experienced

45. Why is Base64 encoding primarily used in Postman?

As a result of the data being transmitted in textual form and being sent in a more accessible format, like HTML form data, the Base64 authorization credentials are typically employed. Base64 is so popular because we can rely on the same 64 characters regardless of the encoding language we use.

46. What is the purpose of the 304 status code?

The meaning of the status code 304 is "Not Modified." It is utilized in conditional GET requests to reduce network bandwidth usage. The response's body must be empty. Dates, locations, and other information should be in the headers.

47. Is it preferable to save our work on Postman Cloud?

Saving your work in the Postman cloud is not recommended because business work cannot be disclosed and must be kept private. Since Postman cloud requires sign-in, security breaches could happen if it gets used. Therefore, using Postman Cloud to save work is discouraged, and team workspace is strongly recommended.

33. What encoding is accepted by Postman in authorization credentials?

Postman only takes authorization credentials encoded in Base64. Postman already has this feature, or you can use a third-party service to convert the credentials to base64.

34. Can we have the same names for global variables in Postman?

Global variables cannot have identical names since doing so would confuse the software because they are devoid of any environment. The terms of local variables might be the same across various environments.

35. What do you know about the postman monitor?

For running collections, the postman monitor is employed. Collections are kept running for the duration of the user-defined time. The user must get logged in to utilize Postman Monitor. Users regularly send emails to one another to share the monitor reports.

36. What is a binary form in POST methods?

Postman's binary form is made to send data in a format that you cannot manually type. You use these options, which cannot be manually typed, such as an image, a file, etc., because everything in a computer is transformed into binary.

37. What are the limitations of Postman?

The following is a list of Postman's main drawbacks:

- Postman cannot process 1000+ API requests.
- It might be challenging to manage collections and requests for large projects.
- For managing the workspace as code, Postman is not appropriate. It is because dynamic API requests would result in a lot of code duplication.

38. How can you save the responses of API to a file in Postman?

There are two ways to save an API response to a file in Postman:

- First, in the response section, click the download button.
- Then, press the arrow next to the send button. An option to send and download is available here. After clicking it, Postman will prompt you for the location to save the response after the request gets successfully executed.

39. What is the significance of 301 status code?

When a page has been permanently redirected from one website page to another, Postman's 301 status code gets used to indicate this. The search engine is informed that the old page is out-of-date and has to index the URL of the new page.

40. Define status code 201.

When you successfully create a resource using a POST or PUT request, the status code 201 denotes that the resource has been created. It uses the location header to return a link to a newly built resource.

41. When do we use global variables, collection variables, and local variables?

Global variables are all-purpose variables perfect for prototyping and quick results. They get utilized when transferring data to other requests.

You can most frequently use collection variables to store constants that don't change while the Collection is being executed. They are used for constants that remain the same throughout execution and for URLs and authentication credentials when just one environment is present.

Local variables are only accessible for the duration of the request that set them or while using

28. What are the various authorization methods provided by Postman?

Postman provides the following API request authorization options:

- API Key
- Basic auth
- Digest auth
- Hawk Authentication
- OAuth 1.0
- OAuth 2.0
- Bearer Token
- NTLM Authentication
- AWS Signature

29. What are the different types of API requests supported in Postman?

The following is a list of the various API requests that Postman supports:

- GET
- POST
- PUT
- PATCH
- COPY
- DELETE
- HEAD
- OPTIONS
- LINK
- UNLINK
- PURGE
- LOCK
- UNLOCK
- PROPFIND
- VIEW

30. How are Query Params different from Path Variables?

Query parameters are used to sort or filter the resources, and Path Variables gets used to identify specific resources.

31. What is Basic Auth in Postman?

Basic Auth is a method of authorization offered by Postman for HTTP user agents, such as web browsers, to enter login and password. It becomes connected with the request after the username and password are entered.

32. What is digest auth in Postman?

One of Postman's authorization techniques is digest auth or digest authorization. Through this method, clients can send requests to the API first and then receive responses from the server, such as 401 illegal responses and numbers that can only be used once as absolute values.

33. What encoding is accepted by Postman in authorization credentials?

Postman only takes authorization credentials encoded in Base64. Postman already has this feature, or you can use a third-party service to convert the credentials to base64.

using it.

56. How can we use Custom Javascript libraries in our scripts with an example?

We may utilize the many built-in tools and libraries that Postman offers to include our pre- or post-request scripts or test cases. Consider the use of the moment.js library. It offers a variety of helpful methods for formatting data around time. Consider a POST request that must provide the generated date for the user, who anticipates the format "DD/MM/YYYY." You can use the moment library with just one line of code. To obtain the data with the proper formatting and then store it in an environment variable, we must add the following lines of code to our pre-request script:

```
var moment = require('moment');

pm.environment.set('createdDate',moment().format('DD/MM/YYYY'));
```

57. If we have a global and a local variable of the same name, which one will be given the most preference in Postman?

In certain circumstances, the global variable's value is overwritten to give the local variable higher precedence.

58. Does Postman allow flexibility to make use of the command-line?

Any Postman collection can get executed using the Newman command-line tool that Postman offers. This [NodeJS](#)-based package uses the Newman Collection Runner to execute collections and therefore needs a node environment. It supports running assertions, pre-request scripts, or other request scripts linked with the requests that are a part of the Collection, just like Postman's Collection Runner does.

59. How will you generate random numbers of a given range in Postman?

Assuming you want to create numbers between the ranges of 1 to N, the pre-request script can be written as follows:

```
pm.globals.set('randomNumber', Math.floor(Math.random() * N));
```

This variable can then be used in the URL as follows:{{randomNumber}}

60. What do you understand about ScratchPad?

Postman allows us to work without a connection to its servers by giving us access to Scratch Pad. It offers the freedom of using some of Postman's functionalities offline. The features include sending requests, creating requests, and adding requests to collections.

61. How do you get the cURL command based on the details of the REST API obtained from Postman?

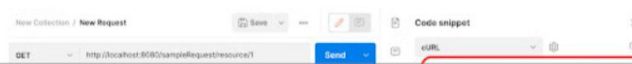
You can use the steps listed below to get the cURL command equivalent:

As indicated below, click the Code icon.



Step 1 - Click on Code

In response to the REST API request, you will receive the cURL command:



```
});
```

51. How do you set the same headers for all requests in a Postman Collection?

Pre-request scripts are supported at the collection level and for individual requests in Postman. Any script that applies to every request in the Collection may get included in the pre-request scripts. The steps are as follows:

To access the pre-request tab, right-click the Collection.

Add the script's lines of code below to add a request header for each request included in the Collection.

```
pm.request.headers.add({  
  
  key: 'TestHeader',  
  
  value: 'testValue'  
  
});
```

To save the script, click Update.

Run the request in the Collection and check the Postman console to ensure that the headers have been added.

52. What are workspaces in Postman? What are their uses?

Postman workplaces are just areas or spaces where one or more people can collaborate on the same Collection or set of collections. It allows the collections or requests to get logically separated from one another. Postman supports two different types of workspaces: Team and Personal.

53. Does Postman provide a feature to log requests and responses?

The Postman software does allow users to view requests and response parameters. But after applying the pre-request scripts, it is essential to see how you sent the request. To inspect every request and response detail in such circumstances, Postman includes an additional tool called "Postman Console." By having console.log statements in the scripts, you can additionally log the information in the console.

54. How will you stop the execution of upcoming requests or the execution of the collections?

You can use the code below to stop processing the next request: `pm.setNextRequest (null);`

55. What do you understand by the pre-request script?

Pre-request scripts are those used to run Javascript code before a request is made. Pre-processing tasks, including setting variables, parameters, headers, body data, etc., are performed using it.

56. How can we use Custom Javascript libraries in our scripts with an example?

We may utilize the many built-in tools and libraries that Postman offers to include our pre- or post-request scripts or test cases. Consider the use of the moment.js library. It offers a variety of helpful methods for formatting data around time. Consider a POST request that must provide the generated date for the user, who anticipates the format "DD/MM/YYYY." You can use the moment library with just one line of code. To obtain the data with the proper formatting and then store it in an environment variable, we must add the following lines of code to our pre-request script:

```
var moment = require('moment');  
  
pm.environment.set('createdDate',moment().format('DD/MM/YYYY'));
```

other information should be in the headers.

47. Is it preferable to save our work on Postman Cloud?

Saving your work in the Postman cloud is not recommended because business work cannot be disclosed and must be kept private. Since Postman cloud requires sign-in, security breaches could happen if it gets used. Therefore, using Postman Cloud to save work is discouraged, and team workspace is strongly recommended.

48. What are the various variable scopes provided by Postman?

Postman provides the following variable scopes:

- Global Variables
- Local Variables
- Environment Variables
- Collection Variables
- Data Variables

49. Is it possible to reuse the authentication token for multiple requests?

You can indeed use the authentication token more than once. To do this, create a collection, add all requests with the same authentication token, and then assign the Collection with the auth token to the Collection. By choosing "Inherit auth from parent" under the Authorization tab, we may apply it to each request separately.

50. How do you write test cases for basic authentication in Postman?

One of the authentication methods offered by Postman, Basic Authentication, ensures we can specify the username and password along with the API calls. We can achieve this by first configuring the API's credentials by:

- Going to the Authorization tab
- From the drop-down option, choose Basic Auth
- Enter the API's username and password in the corresponding fields

You can write the test cases like this:

```
pm.test("Is the Request Authenticated?", function () {

    var jsonData = pm.response.json();

    //if authenticated then assert to true

    pm.expect(jsonData.authenticated).to.eql(true);

});

pm.test("Is the Content-Type present?", function () {

    pm.response.to.have.header("Content-Type");

});

pm.test("Is it a successful POST Request?", function () {

    pm.response.to.have.status(200);

});
```

51. How do you set the same headers for all requests in a Postman Collection?

Pre-request scripts are supported at the collection level and for individual requests in Postman.

Any script that applies to every request in the Collection may get included in the pre-request

[Read More - SOAP API Vs. REST API.](#)

4. Mention Common HTTP Methods used in API Testing.

Here are the Common HTTP Methods used in API Testing:

- **POST:** It is used to create new data or resources on an API.
- **GET:** It is used to retrieve data or resources from an API.
- **PUT:** It is used to update existing data or resources on an API.
- **PATCH:** It is used to update existing data or resources from an API partially.
- **DELETE:** It is used to delete data or resources from an API.

[Read More - HTTP Methods of API Testing.](#)

5. What are the Different Types of API Testing?

Here are the different types of API testing :

- **Functional Testing:** Checks if the API functions correctly according to the requirements.
- **Load Testing:** Tests how the API performs under heavy load or many simultaneous requests to check its stability and responsiveness.
- **Security Testing:** Ensures the API is protected against threats like unauthorized access, injection attacks, and data leaks.
- **Reliability Testing:** Verifies that the API can handle requests consistently over time without failures.
- **Validation Testing:** Confirms that the API returns the expected results and follows the correct data format and protocols.
- **Error Handling Testing:** Checks how the API responds to invalid inputs or unexpected conditions, making sure it handles errors gracefully.
- **Interoperability Testing:** Ensures the API works well across different devices, platforms, or systems that interact with it.
- **Documentation Testing:** Reviews the API documentation to ensure it is accurate, complete, and easy to understand for developers.

6. What is the role of Postman in API testing?

Postman is a tool for designing, testing, and automating API requests. It allows sending requests, validating responses, and scripting tests.

[Read More - Postman for API Development.](#)

7. List the differences between API and Web Service.

Here is the Differences between Web Services and Web API:

Parameters	API	Web Service
Definition	API is an Application Programming Interface that acts as an interface between two applications.	Web services are a type of API that must be accessed through a network connection.
Protocols Support	It provides support for HTTP/s protocol.	It provides support for HTTP protocol.
XML Support	API supports XML and JSON.	Web service supports XML.

[Read More - Web Services Vs. Web API.](#)

8. What protocols can be tested using API Testing?

API testing can be used to test various protocols that facilitate communication between software systems. Some commonly tested protocols include:

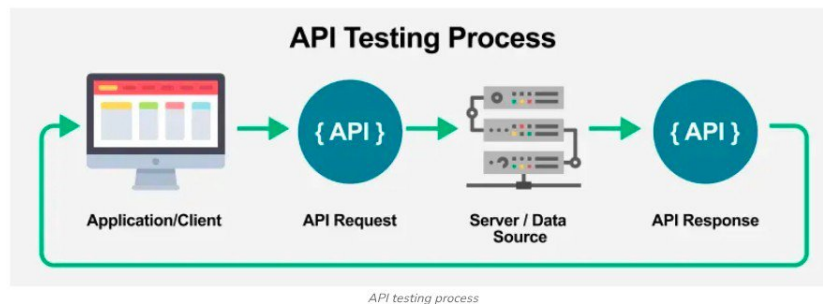
- **HTTP/HTTPS:** Used for RESTful APIs, ensuring secure and structured web communication.
- **SOAP (Simple Object Access Protocol):** A protocol that uses XML-based messaging for web services.
- **REST (Representational State Transfer):** A widely used web service architecture over HTTP.



These questions focus on foundational concepts to help fresher build a strong understanding of API testing basics.

1. What is API testing, and why is it important?

API Testing verifies the functionality, performance, and security of application programming interfaces (APIs) by sending requests and validating responses. It is important because APIs enable communication between systems, and testing ensures the reliability and correctness of the software product.



[Read More - API Testing.](#)

2. What is an API and Types of APIs?

An **Application Programming Interface (API)** is useful for communication between different software systems. It acts as an interface between two applications and allows two software systems to communicate with one another.

APIs (**Application Programming Interfaces**) allow different software systems to communicate with each other.

- 1. Web APIs:** A Web API (also known as Web Service) is one of the most common types of APIs. It works over the internet and follows HTTP protocols. Web APIs are open-source, meaning they can be accessed by multiple clients like phones, tablets, or computers to exchange data.
- 2. Local APIs:** Local APIs allow applications within the same system to communicate with each other. Developers use them for middleware services—software that connects different applications.
- 3. Program APIs:** Program APIs help one software program interact with another, even if they are not on the same system. They often use Remote Procedural Calls (RPCs) to make a remote program appear as if it's running locally.

[Read More - API.](#)

3. What is the difference between REST and SOAP APIs?

- **REST:** Lightweight, uses HTTP methods (GET, POST), stateless, and returns JSON/XML.
- **SOAP:** Protocol-based, uses XML, and supports complex operations with WS-Security.

SOAP API	REST API
Relies on SOAP (Simple Object Access Protocol)	Relies on REST (Representational State Transfer) architecture using HTTP.
Transports data in standard XML format.	Generally transports data in JSON. It is based on URI. Because REST follows a stateless model, REST does not enforce message format as XML or JSON etc.
Because it is XML based and relies on SOAP, it works with WSDL	It works with GET, POST, PUT, DELETE

[Read More - SOAP API Vs. REST API.](#)

4. Mention Common HTTP Methods used in API Testing

Pad. It offers the freedom of using some of Postman's functionalities offline. The features include sending requests, creating requests, and adding requests to collections.

61. How do you get the cURL command based on the details of the REST API obtained from Postman?

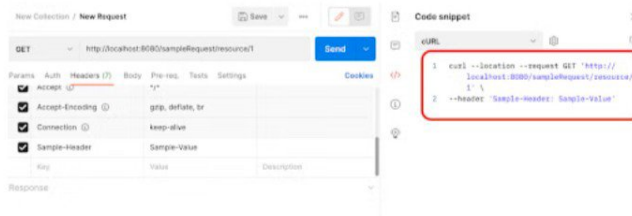
You can use the steps listed below to get the cURL command equivalent:

As indicated below, click the Code icon.



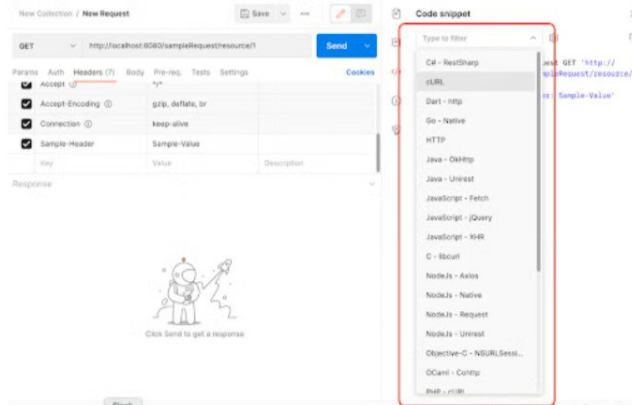
Step 1 - Click on Code

In response to the REST API request, you will receive the cURL command:



Step 2 - REST API request

By choosing the necessary choice from the drop-down, as shown in the image below, we can also obtain the command for the request in several languages, such as C#, Javascript, NodeJs, PHP, etc.



Step 3 - Select Language

Conclusion

Practice these postman interview questions for your next software development interview and get your dream job. Mastering Postman and automation testing is crucial for staying ahead in today's software development landscape. Simplilearn's [Automation Testing Masters Program](#) is a comprehensive training that covers Postman and other essential automation testing tools and techniques. With hands-on projects, expert-led sessions, and real-world scenarios, this program equips you with the skills and knowledge needed to excel in Postman-based API testing roles. Take the next step in your career by enrolling in Simplilearn's Automation Testing Masters Program and unlock exciting opportunities in the world of software testing.

Upskill yourself with our trending [Software Development courses](#) and certifications

1. [Full Stack Java Developer Course](#)

2. [Full Stack Development Course](#)

Query parameters are key-value pairs in the URL to filter or modify a request, appended after a ?.

API Testing Interview Questions for Intermediate

These questions focus on practical automation, scripting, and intermediate concepts for testers.

21. What is API mocking and Why is used ?

API mocking is practice of simulating the behavior of an API endpoint during testing without actually invoking the real API. It is beneficial during the development stage.

- **Request Headers:** Provide metadata about the request, such as authentication (Authorization), content type (Content-Type), and caching control.
- **Response Headers:** Contain metadata about the server response, including status info, content type, and security policies.

Here are the reasons why it is used:

- Mock APIs are useful when applications have dependencies with external APIs.
- Mock APIs are useful for testers to plan and validate test executions and for developers to do unit testing and identify the initial development stages.
- Mock APIs are useful in situations where the complete API needs to be made available for consumer testing before committing it to development stage

22. What is the purpose of HTTP status codes in API testing?

HTTP status codes indicate the result of an API request, helping testers understand whether the request was successful, failed, or requires action.

- **Indicate Response Status:** Shows if the request succeeded (200 OK), failed (400 Bad Request), or needs authentication (401 Unauthorized).
- **Aid Debugging:** Helps identify issues like server errors (500 Internal Server Error).
- **Improve API Communication:** Ensures clear communication between client and server.
- **Enhance Automated Testing:** Used in test validations to verify expected API behavior.

Read More - [HTTP status codes](#).

23. What is the purpose of the request and response body in API testing?

The request body is used to send and receive data such as input parameters, or data to create/update resources via the REST API. The response body is the data API send to the client.

- **Request Body:** Contains data sent by the client to the API (e.g., user credentials, form data, JSON payload). Example: Sending user details in a POST request.
- **Response Body:** Contains data returned by the API, such as success messages, error details, or requested information. Example: Returning user profile details in a GET request.

Read More - [HTTP Request & Responses](#).

24. What is API security testing, and why is it important?

[API security testing](#) is the process of testing the vulnerabilities in the API. This is done through penetration testing or manual scanning of the APIs. API security testing is important:

- Cloud-based applications rely on APIs to exchange data and interact with each other. Any security vulnerabilities can have far-reaching consequences.
- Online operations and businesses rely on APIs to integrate different systems and services. This causes potential security risks.
- Organizations rely on security solutions that are built for web apps to detect and secure from API threats. Such solutions cannot detect unique vulnerabilities and gaps in the APIs.

Read More - [Security Testing](#).

25. What is Boundary Value Analysis (BVA) in API testing?

BVA tests edge cases of input ranges (e.g., min/max values) to find defects at boundaries.

Read More - [BVA](#).

26. What is API versioning, and why is it important in API testing?

returning responses.

13. What is authentication in API testing, its purpose, and common methods?

Authentication verifies the identity of a user or system to access protected API resources, ensuring security and preventing unauthorized access. Its purpose is to safeguard sensitive data and enforce access control. Common methods include:

- **API Keys:** Simple key in headers (e.g., X-API-Key: abc123).
- **OAuth:** Token-based delegation (e.g., OAuth 2.0 for user authorization).
- **JWT:** JSON Web Tokens for secure, stateless access.
- **Basic Auth:** Base64-encoded username:password in headers.

14. What is a test case for API testing, and what does it include?

A test case outlines steps to validate an API, including: Test ID, description, endpoint, method, request data, expected response, and status code.

Read More - [Test Case For API Testing.](#)

15. What is the difference between a GET and POST request?

- **GET:** Retrieves data, idempotent, parameters in URL.
- **POST:** Creates data, non-idempotent, data in request body.

Read More - [GET Vs. POST Request.](#)

16. What is the purpose of headers in API requests?

Headers provide metadata about the request/response, such as content type, authorization, or caching instructions.

17. What is a mock API, and when is it used?

A mock API simulates real API behavior for testing when the actual API is unavailable, using tools like WireMock or Mockoon.

18. What is Negative Testing in the context of APIs?

Negative testing validates API behavior with invalid inputs, such as incorrect data or unauthorized requests, to ensure proper error handling.

Read More - [Negative testing.](#)

19. What is the purpose of authentication in API testing?

API authentication is a combination of technology or process that verifies the identities of users who want access to an API. It involves the use of software protocol to verify identity of the requester before granting access to protected resources.

Authentication verifies the identity of a user or system. Common methods include API keys, OAuth, JWT (JSON Web Tokens), and Basic Auth.

Read More - [API Authentication.](#)

20. What is a query parameter in API testing?

Query parameters are key-value pairs in the URL to filter or modify a request, appended after a ?.

API Testing Interview Questions for Intermediate

These questions focus on practical automation, scripting, and intermediate concepts for testers.

21. What is API mocking and Why is used ?

API mocking is practice of simulating the behavior of an API endpoint during testing without actually invoking the real API. It is beneficial during the development stage.

- **Request Headers:** Provide metadata about the request, such as authentication (Authorization), content type (Content-Type), and caching control.
- **Response Headers:** Contain metadata about the server response, including status info, content type, and security policies.

8. What protocols can be tested using API Testing?

API testing can be used to test various protocols that facilitate communication between software systems. Some commonly tested protocols include:

- **HTTP/HTTPS:** Used for RESTful APIs, ensuring secure and structured web communication.
- **SOAP (Simple Object Access Protocol):** A protocol that uses XML-based messaging for web services.
- **REST (Representational State Transfer):** A widely used web service architecture over HTTP.
- **GraphQL:** A query language for APIs that allows fetching specific data efficiently.
- **WebSockets:** For real-time communication between a client and a server

9. Tell me Advantages and Disadvantages of API Testing.

Advantages:

- **Faster bug fixes:** API testing works quickly to provide a solution, thus it helps to diagnose problems earlier in the development of the system allowing for efficient bug detection and removal.
- **Reduced costs:** API testing provides quick release of results, easier to maintain, thus reducing the cost of testing.
- **Universal language support:** API testing provides support for many different languages like JavaScript, Ruby, Python, and PHP. Formats like XML, and JSON are supported during API testing.

Dis-advantages:

- **Technical skills:** API testing requires skilled and trained testers
- **Time-consuming:** API testing is time-consuming as it requires time to develop and execute test scripts for APIs.
- **Limited documentation:** Limited documentation of the API makes it difficult for the testers to understand how the API should behave in response to a particular input.

10. List the differences between API Testing and Unit Testing.

Here is the Differences between API Testing and Unit Testing:

Parameters	API Testing	Unit Testing
Definition	API testing is used to test the API to ensure that it meets the expectations for functionality, performance, security, etc.	Unit Testing is used to test each unit and to ensure that each unit performs as expected or not.
Carried out	It is carried out by QA team.	It is carried out by developers.
Type testing	It is mostly Black-box testing .	It is White-box testing .

Read More - [API Testing Vs. Unit Testing](#).

11. What is the difference between functional and non-functional API testing?

- **Functional:** Tests API logic and behavior (e.g., correct response data).
- **Non-functional:** Tests performance, security, and scalability (e.g., response time).

12. What is an endpoint in API testing?

An endpoint is a specific URL or route in an API that performs a function, accepting requests and returning responses.

13. What is authentication in API testing, its purpose, and common methods?

Authentication verifies the identity of a user or system to access protected API resources, ensuring security and preventing unauthorized access. Its purpose is to safeguard sensitive data and enforce access control. Common methods include:

- **API Keys:** Simple key in headers (e.g., X-API-Key: abc123).
- **OAuth:** Token-based delegation (e.g., OAuth 2.0 for user authorization).
- **JWT:** JSON Web Tokens for secure, stateless access.
- **Basic Auth:** Base64-encoded username:password in headers.

37. What is API contract testing?

API contract testing is a type of testing that aims to monitor the API conversation that takes place between the API consumer and the API producer. It can be performed using tools like Postman, Swagger, etc.

38. Why API contract testing important?

API contract testing ensures that APIs follow a defined structure (contract) between the client and server, preventing breaking changes.

- **Ensures Consistency:** Verifies API responses match the expected format (JSON, XML).
- **Detects Breaking Changes:** Prevents issues when updating APIs.
- **Improves Reliability:** Ensures backward compatibility across services.
- **Speeds Up Development:** Allows frontend and backend teams to work independently.
- **Enhances API Stability:** Reduces failures in integrations by enforcing standards.

39. What is API performance testing?

API Performance Testing evaluates an API's speed, scalability, and reliability under different loads and conditions. It ensures that the API responds efficiently and remains stable under varying traffic levels.

- **Response Time:** Measures how quickly the API responds.
- **Throughput:** Checks the number of requests handled per second.
- **Load Testing:** Simulates high user traffic to check performance.
- **Stress Testing:** Determines API behavior under extreme conditions.
- **Spike Testing:** Evaluates API response to sudden traffic surges.

40. Why API performance testing important?

API performance testing ensures that an API is fast, scalable, and reliable under different workloads.

- API performance testing helps to improve the API's overall performance and stability.
- It helps to provide insight into the API's overall performance thus helping to identify the areas of strength and weakness.
- It helps to identify the performance issues and determine the impact of changes.
- It helps to ensure that the API is flexible and can handle the demands of the real world.

41. What is API monitoring?

API monitoring is the process of continuously tracking an API's availability, performance, and functionality to detect issues in real-time.

- **Uptime & Downtime:** Ensures the API is always accessible.
- **Response Time:** Tracks API speed and latency.
- **Error Rates:** Detects failed requests or unexpected responses.
- **Traffic & Load:** Monitors API usage patterns.
- **Security Threats:** Identifies potential vulnerabilities.

Read More - [API monitoring](#).

42. Why API monitoring important?

API monitoring ensures that APIs remain reliable, fast, and secure by tracking their performance in real-time.

- They provide measurements of how long a routine takes to execute, how often it is called, and how much of total time is spent in executing the transaction.
- It is important to ensure availability, performance, and security of the APIs.
- It helps to guarantee a dependable and effective user experience.
- It helps to track the availability of the critical APIs.

43. What is API virtualization?

API virtualization is the process of using a tool that creates a virtual copy of the API mirroring all of the specifications of the production API and using this virtual copy for testing.

- Allows early testing before the actual API is built.
- Reduces dependency on live systems by simulating unavailable APIs.
- Improves development speed by enabling teams to work in parallel.
- Supports performance testing under various conditions.

31. What are the different types of error responses in API testing?

Error responses in API testing are categorized using HTTP status codes to indicate different types of failures. Here are the different types:

- **Validation errors:** Validation errors occur when the API request does not meet the validation criteria defined by the API.
- **HTTP error status codes:** These can be client-side status codes or server-side status codes. There are standard HTTP status codes like 4xx or 5xx that indicate errors in the API request or response.
- **Custom error messages:** These are the customer error messages that are returned by the API in case of errors or exceptions.

32. How do you handle error responses in your API tests?

Handling Error Responses in API Testing:

- **Validate HTTP Status Codes:** Ensure correct 4xx (client errors) and 5xx (server errors) are returned.
- **Check Error Messages:** Verify meaningful, user-friendly error messages.
- **Test Edge Cases:** Send invalid inputs, missing parameters, and malformed requests.
- **Implement Retry Mechanism:** Retry requests for temporary failures (5xx errors).
- **Monitor & Log Errors:** Use tools like Postman, JMeter to track failures.
- **Respect Rate Limits (429):** Implement delays or follow Retry-After headers.
- **Secure Error Handling:** Ensure no sensitive data is leaked in error responses.

33. What is input validation?

Input validation is the process of verifying user input to ensure it is correct, secure, and meets expected criteria before processing. It helps prevent errors, data corruption, and security vulnerabilities like SQL Injection and XSS attacks. Input validation can be performed on both client-side and server-side for better security and data integrity.

34. Why input validation is important in API testing?

Input validation is important in API testing as it prevents malicious data from entering the system. It is important to prevent injection attacks and other malicious activities. This can be done manually or using automated tools.

35. What is cross-site request forgery (CSRF)?

Cross-site request forgery is an attack that forces user to execute unwanted action in the web application in which they are authorized to perform actions. This attack exploits the trust the web application has in an authenticated user.

Read More - [Cross-site request forgery](#).

API Testing Interview Questions for Experienced

These questions focus on Experienced concepts to help build a strong understanding of API testing with advance concepts.

36. How CSRF can be prevented in API testing?

- **CSRF tokens:** CSRF can be prevented by using CSRF tokens. These are the random tokens that need to be unique per user session and should be of large random value to make it difficult to guess.
- **SameSite cookies:** SameSite is a browser security mechanism that determines when a website's cookies are included in the requests originating from other websites.
- **Referer-based validation:** HTTP referer header can be used to defend against CSRF attacks by verifying the request originated from the application's own domain.

37. What is API contract testing?

API contract testing is a type of testing that aims to monitor the API conversation that takes place between the API consumer and the API producer. It can be performed using tools like Postman, Swagger, etc.

38. Why API contract testing important?

API contract testing ensures that APIs follow a defined structure (contract) between the client and server, preventing breaking changes.

Read More - [Security Testing](#).

25. What is Boundary Value Analysis (BVA) in API testing?

BVA tests edge cases of input ranges (e.g., min/max values) to find defects at boundaries.

 Read More - [BVA](#).

26. What is API versioning, and why is it important in API testing?

API versioning involves specifying a version number in the API endpoint or headers to ensure the backward compatibility. It manages the changes in the API over time.

- It ensures the correct version of the API is being tested.
- It ensures that the changes in the API do not break the existing client applications.

27. What tools could be used for API testing?

- **Katalon:** This is easy-to-use tool that supports REST, SOAP requests, and SSL client certificates. It also enables test import from Swagger, Postman, WADL, and WSDL.
- **JMeter:** Apache JMeter is an open-source, 100% Java application that is created for performance testing. This tool enables automatic working with CSV files.
- **REST-assured:** REST-assured is an open-source Java domain-specific language that enables testing REST services. It supports POST, GET, PUT, DELETE, PATCH, and HEAD requests.
- **SoapUI:** SoapUI is an automated testing tools for Soap and REST APIs. It is an open-source tool that allows to create tests effortlessly with drag and drop, point-and-click.
- **Postman:** Postman is an application that is used for API testing. It is a standalone platform that is used to build, test, design, modify, and document APIs.

28. When writing API document, what must be considered?

Key Considerations When Writing API Documentation

- **Clear API Overview:** Explain API purpose, features, and use cases.
- **Endpoint Details:** List all endpoints (GET /users, POST /orders) with descriptions.
- **Request & Response Format:** Provide JSON/XML examples with expected parameters.
- **Authentication & Security:** Explain how to use API keys, OAuth, or JWT for access.
- **Error Handling:** Document status codes (400 Bad Request, 500 Internal Server Error) and responses.
- **Rate Limits & Throttling:** Define request limits and handling of 429 Too Many Requests.

29. How do you test GraphQL APIs compared to REST APIs?

- **GraphQL Testing:** Focuses on queries, mutations, and schema validation. Use tools like Postman or Apollo Client to send queries (e.g., { user(id: 1) { name } }) and verify response structures. Test for over-fetching/under-fetching and schema compliance.
- **REST Testing:** Tests fixed endpoints (e.g., GET /users/1) with predictable responses. Focus on HTTP methods and status codes.

30. How do you ensure the security of APIs in your tests?

To secure APIs during testing, follow these best practices:

- **Authentication & Authorization:** Verify security mechanisms like: OAuth 2.0, [JWT](#) (JSON Web Token), API Keys, Basic Auth
- **Input Validation & Sanitization:** Prevent [SQL Injection](#) and [XSS](#) attacks by validating input data.
- **Secure Headers & Encryption:** Use HTTPS (TLS/SSL) and set security headers like: [Strict-Transport-Security](#), [Content-Security-Policy](#)
- **Rate Limiting & Throttling:** Prevent DDoS attacks by limiting API request rates.
- **Token Expiry & Refresh Mechanism:** Ensure JWTs expire and implement refresh tokens securely.
- **Access Control (RBAC):** Restrict API access based on user roles and permissions.
- **Logging & Monitoring:** Track API activity using logging tools for anomaly detection.
- **Automated Security Testing:** Use tools like [OWASP ZAP](#), Postman Security Tests, Burp Suite to identify vulnerabilities.

31. What are the different types of error responses in API testing?

Error responses in API testing are categorized using HTTP status codes to indicate different types of failures. Here are the different types:

- **Validation errors:** Validation errors occur when the API request does not meet the validation criteria defined by the API.
- **HTTP error status codes:** These can be client-side status codes or server-side status codes.

43. What is API virtualization?

API virtualization is the process of using a tool that creates a virtual copy of the API mirroring all of the specifications of the production API and using this virtual copy for testing.

- Allows early testing before the actual API is built.
- Reduces dependency on live systems by simulating unavailable APIs.
- Improves development speed by enabling teams to work in parallel.
- Supports performance testing under various conditions.

44. Why API virtualization is used in API testing?

- It is used in API testing as it allows for testing much earlier in the development process, removing the key bottlenecks that would otherwise delay production.
- It is helpful to isolate dependencies, simulate responses, and ensure consistent behavior.

45. What is the purpose of API documentation in API testing?

API documentation is a technical document that describes the API in detail. It includes instructions on how to effectively use and integrate API and provides updates regarding API's lifecycle such as new versions. It is primary resource explaining what is possible with the API and how to get started with the API.

46. How do you prioritize API test cases for regression testing?

- **Critical Functionality First:** Focus on core APIs that impact business operations.
- **Frequently Used APIs:** Test APIs that are accessed most often by users.
- **Recently Changed or Updated APIs:** Prioritize APIs modified in recent updates.
- **APIs with High Bug History:** Retest APIs that previously had defects.
- **Security and Authentication APIs:** Ensure no vulnerabilities are introduced.
- **Performance-Critical APIs:** Validate APIs affecting system speed and scalability.
- **Dependent APIs:** Test APIs that interact with multiple services or components.

47. How to handle versioning in API testing?

- **Test Multiple Versions:** Ensure backward compatibility by testing old and new API versions.
- **Use Version-Specific Endpoints:** Validate API calls with versioned URLs (e.g., `/v1/users` vs. `/v2/users`).
- **Check Header-Based Versioning:** Verify API responses when versioning is done via headers (Accept: `application/vnd.api.v2+json`).
- **Compare Responses:** Ensure consistency between different versions while testing improvements.
- **Validate Deprecated Features:** Test error handling for removed or updated functionalities.
- **Automate Version-Based Tests:** Maintain separate test suites for each API version.

48. List the challenges faced while performing API testing.

- **Building irrelevant tests:** Building tests without considering how the APIs will be consumed may be quicker in short term.
- **Initial setup of API testing:** Setting up an API testing setup requires certain level of expertise and dedication among the team members.
- **Not including API dependencies:** Failure to include API dependencies as a part of API testing strategy can be a critical API testing challenge.
- **Not validating data:** It might be possible that API tests pass successfully but APIs are not returning the correct data in their responses.

49. What are the different bugs that can be found in API testing?

Different Bugs Found in API Testing:

- Duplicate or missing functionality.
- Improper messaging.
- Multi-threading issues.
- Security and performance issues.
- Reliability issues.

50. Is it possible to hack API while testing?

Yes, it is possible to hack API while testing as requests are being sent over the internet which mostly follows HTTP protocol which is a text-based protocol. Hence, it is important to perform security testing of the APIs to ensure safer systems.

Common API Vulnerabilities Exploited:



- Security and performance issues.
- Reliability issues.

50. Is it possible to hack API while testing?

Yes, it is possible to hack API while testing as requests are being sent over the internet which mostly follows HTTP protocol which is a text-based protocol. Hence, it is important to perform security testing of the APIs to ensure safer systems.

Common API Vulnerabilities Exploited:

- **Broken Authentication:** Weak tokens or missing authentication.
- **SQL Injection:** Malicious queries to access or modify databases.
- **Cross-Site Scripting (XSS):** Injecting scripts into API responses.
- **Insecure Endpoints:** Exposed sensitive data due to improper access controls.
- **Rate Limiting Bypass:** Exploiting APIs by sending excessive requests.

51. How do you perform API Load Testing?

[API load testing](#) is a type of performance testing that is done to check the application's capability to perform under various user loads. It is done by simulating many users hitting the API at the same time to identify if the application is capable of handling the load by maintaining the consistency in the response times and not impacting the functionality.

Read More - [API Load Testing](#).

52. What is the test environment of API?

The API test environment is a setup where APIs are tested under controlled conditions before deployment. It includes:

- **Server & Database:** A dedicated test server and database to mimic production.
- **API Endpoints:** Staging or sandbox URLs separate from live systems.
- **Authentication & Security Setup:** API keys, OAuth, JWT tokens for access control.
- **Mock Services & Virtualization:** Simulated dependencies for testing APIs in isolation.
- **Testing Tools:** Tools like Postman, JMeter, SoapUI for automation and performance testing.
- **Logging & Monitoring:** Tracks API requests, responses, and failures.

Practical API Testing Interview Questions

Below are intermediate-level, hands-on API testing questions with detailed solutions, code snippets, and outputs, tailored for SDETs and testers preparing for interviews.

1. Write a RestAssured script to validate a POST API for creating a user, including response schema and status code.

Steps:

- Set up RestAssured with the base URI.
- Send a POST request with a JSON payload containing name and job.
- Validate the status code and response fields.
- Use a JSON schema to ensure response structure.
- Extract and print the user ID.

```
import io.restassured.RestAssured;
import io.restassured.module.json.JsonSchemaValidator;
import io.restassured.response.Response;
import org.testng.Assert;
import org.testng.annotations.Test;

public class PostApiTest {
    @Test
    public void testPostUser() {
        // Set base URI
        RestAssured.baseURI = "https://reqres.in/api";

        // Define request payload
        String requestBody = "{\"name\": \"Alice\", \"job\": \"Engineer\"}";

        // Send POST request and validate
        Response response = RestAssured.given()
            .header("Content-Type", "application/json")
            .body(requestBody)
            .when()
            .post("/users")
            .then()
            .statusCode(201)
            .body("name", org.hamcrest.Matchers.equalTo("Alice"))
            .body("job", org.hamcrest.Matchers.equalTo("Engineer"))
            .body(JsonSchemaValidator.matchesJsonSchemaInClasspath("user-schema.json"))
            .extract()
            .response();

        // Extract and print user ID
        String userId = response.jsonPath().getString("id");
        System.out.println("Created User ID: " + userId);
    }
}
```